

Hands-on 4

Difference Between JPA, Hibernate, and Spring Data JPA

1. Java Persistence API (JPA)

JPA stands for Java Persistence API. In simple words, a specification is just a document that says 'this is how things should work' — but it does not actually build the thing itself.

JPA tells us how to persist (save), read, update, and delete Java objects in a database. It defines things like how to write queries, how to map a Java class to a database table, and how transactions should be handled. But JPA itself has no real working code — it is just a set of interfaces and guidelines.

Key Points about JPA:

- JPA is only a specification — it defines the rules but does not run any code on its own.
- It does not connect to any database by itself.
- You always need an implementation tool like Hibernate to use JPA in a real project.
- Other JPA implementations also exist, like EclipseLink and OpenJPA, but Hibernate is the most widely used.
- JPA gives us annotations like `@Entity`, `@Table`, `@Column`, and `@Id` to map Java classes to database tables.

A simple real-life comparison: JPA is like a set of traffic rules. The rules say how vehicles should behave on the road, but the rules themselves do not drive the car. You need an actual driver (like Hibernate) to follow those rules and drive.

2. Hibernate

Hibernate is an ORM tool — ORM stands for Object Relational Mapping. It is the actual working tool that follows the JPA specification and connects your Java code to the database. When you use Hibernate, it automatically handles the work of converting Java objects into database rows, and database rows back into Java objects.

Before tools like Hibernate existed, developers had to write a lot of SQL queries manually and manage the database connection themselves. Hibernate took away most of that manual work. You just define your Java class, map it to a table using annotations, and Hibernate handles the SQL in the background.

Key Points about Hibernate:

- It is a real, working implementation of the JPA specification.
- It generates SQL queries automatically based on your Java objects.
- You still need to manage sessions, transactions, and exception handling yourself when using Hibernate directly.
- It gives you a lot of control over how your data is fetched and saved.
- Hibernate also has some features beyond JPA — like its own Session API and caching system.

- It supports Lazy Loading — meaning it only fetches related data from the database when you actually need it, which saves time and memory.

A simple real-life comparison: If JPA is the traffic rulebook, Hibernate is the driver who knows and follows all those rules. It actually does the driving — connecting to the database, running queries, and returning results.

3. Spring Data JPA

Spring Data JPA is built on top of JPA and works with Hibernate underneath. It does not replace Hibernate — it actually uses it. The main purpose of Spring Data JPA is to make things even simpler for the developer by removing most of the repeated and boring code that you normally have to write.

When you use Spring Data JPA, you just need to create a simple interface that extends `JpaRepository`. Spring automatically provides all the basic methods like `save()`, `findById()`, `findAll()`, `delete()`, and more — without you having to write a single line of SQL or session management code.

Key Points about Spring Data JPA:

- It does not have its own JPA implementation — it uses Hibernate (or another provider) behind the scenes.
- It sits as an extra layer on top of JPA to make development faster and cleaner.
- It removes boilerplate code — you do not have to open sessions, manage transactions, or write try-catch blocks manually.
- Transaction handling is automatic — just add the `@Transactional` annotation.
- You can even write custom queries using method names — for example, `findByName()` or `findByDepartmentAndSalaryGreaterThan()` — without writing any SQL.
- It is the most common way to work with databases in Spring Boot projects today.

A simple real-life comparison: If Hibernate is the driver, Spring Data JPA is like hiring a chauffeur service. You just say where you want to go (what data you need), and everything else is handled for you automatically.

6. How They All Work Together

In a Spring Boot project, all three are present at the same time. They each play their own role and work together as a stack, one on top of the other.

Step	Layer	What it does
1	JPA	Sets the rules — defines annotations like <code>@Entity</code> , <code>@Id</code> , <code>@Column</code> and how persistence should work
2	Hibernate	Follows the JPA rules, generates SQL, and actually talks to the database
3	Spring Data JPA	Sits on top of Hibernate, provides ready-made repository methods, and removes all the manual session and transaction code

8. Summary

To put it all together in simple words:

- JPA is a set of rules (like a contract or a specification). It tells tools how they should handle data persistence in Java, but it does not do any work by itself.
- Hibernate is a tool that follows those JPA rules and actually does the work — it connects to the database, runs the SQL, and maps data to Java objects.
- Spring Data JPA makes using Hibernate even easier. It sits on top of Hibernate and removes most of the code a developer would normally have to write manually.

The three of them work together like layers in a stack. JPA at the bottom sets the contract, Hibernate in the middle does the actual database work, and Spring Data JPA at the top makes the developer's job much easier.

For any developer starting out with Spring Boot, Spring Data JPA is the most practical and beginner-friendly option. As you grow more experienced, understanding Hibernate and JPA underneath will help you solve more complex problems and debug issues more effectively.

9. Key Annotations to Know

Here are some important annotations that are commonly used when working with JPA, Hibernate, and Spring Data JPA. These will appear in almost every project.

Annotation	Belongs To	What it does
@Entity	JPA	Marks a Java class as a database table
@Table	JPA	Specifies the name of the database table to map to
@Id	JPA	Marks a field as the primary key of the table
@Column	JPA	Maps a field to a specific column in the table
@GeneratedValue	JPA	Tells JPA to auto-generate the primary key value
@Transactional	Spring	Automatically handles commit and rollback for a method
@Autowired	Spring	Injects a dependency (like a repository) automatically
@Repository	Spring	Marks an interface or class as a data access layer component